План разработки AI Stock Market

Версия: 1.0

Дата: 2026-10-14

Статус: спецификация; реализация не начата

Репозиторий: https://github.com/Zaywoo66/AI-Stock-market

1. Цель и исходная точка

Создать веб-приложение для наблюдения за акциями: рыночный дашборд, регистрация, поиск и страницы компаний, личный список отслеживания, ежедневная AI-сводка и уведомления о заданных изменениях цены и объёма торгов. Интерфейс должен работать на компьютере и телефоне.

Репозиторий на момент подготовки плана пуст. Архитектурных решений и готового кода, которые нужно сохранять, пока нет.

План основан на видеоуроке JavaScript Mastery, его автосубтитрах и публичном коде автора. В уроке последовательно рассматриваются настройка (00:03), интерфейс и графики (00:16-01:09), регистрация и БД (01:09-02:54), ежедневная рассылка (02:54-03:38), поиск и акции (03:38-04:07), развёртывание (04:07-04:13). Список отслеживания предложен как отдельное задание в 04:13. Поэтому этапы списка и ценовых уведомлений ниже описаны как самостоятельная разработка, а не как уже готовый результат урока.

1.1 Границы первой версии

Входит: просмотр рынка и отдельных акций, аккаунт, поиск, список отслеживания, ежедневная сводка по новостям, ценовые уведомления по правилам пользователя, настройки уведомлений, развёртывание.

Не входит: торговля, пополнение счёта, хранение денег, инвестиционные рекомендации, гарантированный поток биржевых сделок с минимальной задержкой, админ-панель и прогноз доходности. Эти функции требуют отдельных требований и решений по данным и правам доступа.

1.2 Принципы продукта

- Показатели и новости сопровождаются источником и временем обновления.
- AI пересказывает только полученные новости; не выдумывает цены, события или советы купить/продать.
- Ошибка внешнего API не ломает всю страницу: пользователь видит понятное состояние ошибки и время последнего успешного обновления.
- Пользователь управляет рассылкой и может отключить её.
- Частота уведомлений соответствует фактической доступности данных и ограничениям провайдера. Слово «мгновенно» не используется без подтверждённого источника событий и измеренной задержки.

2. Предлагаемая архитектура

- Интерфейс и сервер: Next.js App Router, TypeScript, Tailwind CSS, shadcn/ui.
- Аутентификация: Better Auth, сессии на сервере.
- Хранение: MongoDB; отдельные коллекции профилей, списков, правил уведомлений и истории отправок. Схемы и индексы описаны в этапе 2.
- Рыночные данные и новости: Finnhub через серверный адаптер; ключ API не передавать браузеру.
- Графики и рыночные виджеты: TradingView. Виджеты отображают данные TradingView; их обновление не следует путать с собственной системой событий приложения.

- Фоновые задания: Inngest для рассылок и проверок условий уведомлений.
- AI: Gemini для кратких новостных сводок с проверяемыми ссылками на исходные материалы.
- Email: транзакционный почтовый сервис через единый интерфейс отправки; Nodemailer допустим для локальной разработки и при выбранном SMTP-провайдере.
- Развёртывание: Next.js-хостинг, MongoDB и Inngest в совместимой среде; конкретный провайдер фиксируется до этапа 8.

2.1 Потоки данных

1. Браузер запрашивает страницу. Сервер проверяет сессию и отдаёт данные пользователя.
2. Поиск и страница акции обращаются к серверному адаптеру Finnhub; адаптер валидирует символ, кэширует ответ в допустимых пределах и возвращает нормализованную модель.
3. Пользователь добавляет символ в список или создаёт правило. Сервер повторно проверяет сессию и записывает изменение в БД.
4. Фоновое задание читает список/правила, получает свежие данные, формирует сводку или проверяет порог, записывает результат и отправляет email.
5. Каждая отправка имеет идентификатор события для защиты от дублей и диагностики.

2.2 Минимальная модель данных

- User/Profile: идентификатор, email, имя, страна, инвестиционная цель, допустимый риск, предпочитаемая отрасль, часовой пояс, согласие на письма. Пароль и сессии принадлежат Better Auth.
- WatchlistItem: userId, нормализованный symbol, companyName, createdAt; уникальный индекс (userId, symbol).
- AlertRule: userId, symbol, тип (price_above, price_below, позднее volume_above), порог, статус, время создания, последнее срабатывание и период подавления повторов.
- NotificationDelivery: userId, тип, символ, ключ события, время, статус, код ошибки. Уникальный ключ события предотвращает повторную отправку.
- DigestPreference: включено/выключено, частота, часовой пояс, время отправки, последний успешно отправленный выпуск.

Схему уточнить перед миграциями; не хранить в БД ключи внешних сервисов и содержимое секретов.

3. Очерёдность и результаты

Этап 0. Уточнение продукта и подготовка (1-2 дня)

Цель: зафиксировать решения, от которых зависят API, уведомления и приёмка.

Задачи

- P0.1. Определить рынки и биржи первой версии, базовую валюту, язык интерфейса и целевую аудиторию.
- P0.2. Определить требуемую свежесть цен на странице и допустимую задержку уведомлений. Проверить, что выбранный тариф и лицензия на данные позволяют нужное использование.
- P0.3. Выбрать хостинг, почтовый сервис, среду БД и владельцев учётных записей. Зафиксировать квоты и предполагаемое число пользователей.
- P0.4. Утвердить основные экраны: вход, регистрация, дашборд, поиск, акция, список, правила, настройки.
- P0.5. Согласовать критерии приёмки первой версии и примерный набор тикеров для проверки.

Результат: короткий docs/product-decisions.md с принятыми решениями и открытыми вопросами; перечень сервисов и переменных окружения без значений секретов.

Готово, когда: нет неизвестного решения, которое блокирует этапы 1-3; требуемая задержка уведомлений сформулирована числом или диапазоном.

Этап 1. Каркас приложения и качество сборки (1-2 дня)

Зависимость: решения P0.1 и P0.4.

Задачи

- P1.1. Создать Next.js App Router с TypeScript и закреплёнными совместимыми версиями зависимостей.
- P1.2. Подключить Tailwind CSS, базовые компоненты shadcn/ui, шрифты, тему и адаптивный каркас страниц.
- P1.3. Разделить серверный и клиентский код; добавить типы доменных моделей и единый модуль конфигурации.
- P1.4. Создать .env.example, .gitignore, команды dev, build, lint, typecheck; настроить автоматические проверки для новых изменений.
- P1.5. Добавить пустые состояния, общие страницы ошибок и базовую доступность навигации.

Результат: рабочий каркас с маршрутизацией и понятным запуском по README.

Готово, когда: проект запускается по инструкции; build, lint и проверка типов проходят; в Git нет секретов и локальных артефактов.

Этап 2. Учётные записи и профиль (2-3 дня)

Зависимость: этап 1, выбранная MongoDB.

Задачи

- P2.1. Подключить MongoDB и Better Auth, настроить регистрацию по email/паролю, вход, выход и серверную проверку сессии.
- P2.2. Создать анкеты страны, целей, риска и отрасли; хранить предпочтения отдельно от чувствительных данных аутентификации.
- P2.3. Защитить личные маршруты и серверные операции; исключить доступ к чужим спискам и правилам через изменение идентификатора.
- P2.4. Обработать ошибки валидации, дубли email, истёкшие сессии и восстановление доступа в согласованном объёме.
- P2.5. Добавить настройки согласия на ежедневные письма и часового пояса.

Результат: пользователь может зарегистрироваться, войти, изменить допустимые настройки и выйти.

Готово, когда: перезагрузка страницы сохраняет сессию; чужие записи недоступны; формы показывают понятные ошибки; email-предпочтения сохраняются.

Этап 3. Дашборд и рыночные данные (2-3 дня)

Зависимость: этапы 1-2, доступ к TradingView и Finnhub.

Задачи

- P3.1. Собрать главную страницу: обзор индексов, карта рынка, основные котировки, новости, состояния загрузки и ошибки.
- P3.2. Интегрировать TradingView-виджеты с контролируемыми настройками символов, высоты, темы и адаптивности.
- P3.3. Добавить серверный клиент Finnhub: таймаут, обработка квот и ошибок, нормализация цены/времени/валюты, кэширование.
- P3.4. Показывать источник и время обновления там, где приложение отображает собственные данные.

- P3.5. Проверить мобильную вёрстку и работу при недоступности одного из поставщиков.

Результат: дашборд открывается после входа и показывает рыночную картину.

Готово, когда: виджеты читаемы на узком экране; отказ Finnhub или отдельного виджета не превращает весь экран в ошибку.

Этап 4. Поиск и страница акции (2-3 дня)

Зависимость: этап 3.

Задачи

- P4.1. Реализовать поиск по символу и названию с задержкой ввода, отменой устаревших запросов и ограничением частоты.
- P4.2. Нормализовать и валидировать символы, показать биржу и тип инструмента в результатах.
- P4.3. Создать маршрут /stocks/[symbol]: цена и изменение, графики, профиль компании, основные финансовые показатели и новости, если данные доступны.
- P4.4. Сделать понятные состояния «символ не найден», «данные недоступны» и «квота исчерпана».
- P4.5. Связать результаты поиска со страницей акции и будущим списком отслеживания.

Результат: пользователь находит акцию и открывает её страницу.

Готово, когда: поиск работает по названию и тикеру; неизвестный тикер не вызывает ошибку сервера; запросы к Finnhub идут только с сервера.

Этап 5. Личный список отслеживания (2-3 дня)

Зависимость: этапы 2 и 4. Это отдельная реализация финального задания из видео.

Задачи

- P5.1. Добавить модель WatchlistItem и уникальный индекс (userId, symbol).
- P5.2. Создать серверные операции добавления и удаления с проверкой сессии, символа и владельца записи.
- P5.3. Подключить звёздочку в поиске и кнопку на странице акции; состояние должно приходить из БД, а не только из локального React-состояния.
- P5.4. Создать страницу списка: символ, компания, актуальная цена, изменение, дата добавления; сортировка и пустое состояние.
- P5.5. Обработать повторный клик и сбой API без потери согласованности интерфейса и БД.

Результат: персональный список, пригодный для рассылки и уведомлений.

Готово, когда: список переживает выход и повторный вход; пользователи видят только свои записи; дубль не создаётся.

Этап 6. Ежедневная AI-сводка (2-4 дня)

Зависимость: этапы 2, 4 и 5; доступ к Inngest, Gemini и почтовому сервису.

Задачи

- P6.1. Подключить Inngest и создать задания приветственного письма и ежедневной сводки.
- P6.2. Для каждого подписанного пользователя получить новости по его списку; если список пуст, использовать общий рынок.
- P6.3. Передавать AI только нормализованные заголовки, ссылки, даты и краткие тексты; требовать фактического пересказа без выдуманных чисел и торговых советов.

- P6.4. Формировать письмо с датой, источниками и ссылкой на настройки; экранировать или санитизировать вывод AI перед вставкой в HTML.
- P6.5. Учитывать часовой пояс и согласие пользователя; обеспечить повторные попытки, защиту от повторной отправки и журнал результата.
- P6.6. Предусмотреть ручной тестовый запуск на одном тестовом аккаунте и возможность отключить расписание.

Результат: автоматическое персонализированное письмо с новостями отслеживаемых компаний.

Готово, когда: тестовый пользователь получает один выпуск; отписанный не получает письмо; сбой одного адреса не останавливает рассылку остальным.

Этап 7. Правила уведомлений (3-5 дней)

Зависимость: этапы 3, 5 и 6; решение P0.2 о задержке и квотах данных.

Задачи

- P7.1. Добавить модель AlertRule и экран создания, изменения, приостановки и удаления правил.
- P7.2. Реализовать условия price_above и price_below с указанием валюты, источника цены и момента проверки.
- P7.3. Выбрать механизм проверки: провайдерские события, если доступны на выбранном плане, или периодические задания в пределах квот. Частоту документировать в интерфейсе.
- P7.4. Отправлять email при переходе через порог; записывать ключ события, последнее срабатывание и период подавления повторов.
- P7.5. Добавить volume_above только после проверки доступности и смысла данных об объёме для выбранных рынков.
- P7.6. Проверить нерабочие дни биржи, отсутствие котировки, повтор фонового задания и изменение правила между проверками.

Результат: управляемые пользователем уведомления с измеримой задержкой.

Готово, когда: заданный тестовый переход через порог отправляет одно письмо; повтор того же события не создаёт дубль; правило можно выключить.

Этап 8. Проверка, эксплуатация и выпуск (2-3 дня)

Зависимость: этапы 1-7.

Задачи

- P8.1. Проверить полный путь: регистрация -> поиск -> добавление -> сводка -> правило -> уведомление -> отписка.
- P8.2. Добавить автоматические проверки ключевой бизнес-логики: изоляция данных пользователей, уникальность списка, порог/повторы, выбор адресатов рассылки.
- P8.3. Проверить сборку, адаптивность, клавиатурную навигацию, ошибки API и ограничения квот.
- P8.4. Настроить production-переменные, домен, фоновые задания, логирование ошибок и мониторинг отправки писем.
- P8.5. Задокументировать запуск, восстановление после сбоя, ротацию ключей и отключение рассылок.

Результат: доступный по рабочему адресу продукт с проверенными основными сценариями.

Готово, когда: контрольный сценарий проходит в production; ключи не раскрыты; есть журнал ошибок и способ остановить фоновые задания.

4. Вехи, сроки и приоритет

Оценка для одного разработчика при готовых доступах к сервисам: 16-25 рабочих дней. Это ориентир, который уточняется после этапа 0.

- **M1 - просматриваемый прототип:** этапы 0-3. Есть каркас, аккаунт и дашборд.
- **M2 - полезная первая версия:** этапы 4-6. Есть поиск, страницы акций, постоянный список и ежедневная сводка.
- **M3 - полный аналог заявленных функций:** этапы 7-8. Есть правила уведомлений, проверка и выпуск.

Если сроки ограничены, выпускать M2 отдельно. Ценовые уведомления нельзя обозначать готовыми до выполнения P7.1-P7.6 и проверки задержки на реальном источнике данных.

5. Сквозные критерии готовности

Каждая задача завершена, только если:

1. Изменение привязано к пункту плана и имеет проверяемый результат.
2. Секреты остаются вне репозитория; публичные переменные не содержат приватных API-ключей.
3. Доступ к личным данным проверяется на сервере для каждого чтения и изменения.
4. Ошибки поставщиков данных, AI и почты обрабатываются и записываются без раскрытия секретов.
5. Добавлены необходимые проверки поведения, а сборка, линтер и типы проходят.
6. Обновлены README, .env.example и документы, если изменились запуск, данные или контракты.
7. Агент или разработчик передал результат по шаблону agents/contracts/handoff.md.

6. Решения, которые нужно принять до реализации

- D1 — Какие рынки поддерживать? — Начать с акций США; расширение после проверки форматов тикеров и покрытия API.
- D2 — Как быстро обновлять цены и правила? — Установить измеримый интервал после проверки плана провайдера; показывать его пользователю.
- D3 — Когда отправлять сводку? — Настраиваемое локальное время пользователя; один выпуск в календарный день его часового пояса.
- D4 — Как отправлять письма? — Выбрать транзакционный сервис с журналом доставки; SMTP через Nodemailer для разработки допустим.
- D5 — Где размещать продукт? — Выбрать после проверки бюджета, региона и совместимости фоновых заданий.

До принятия решения агент фиксирует предположение в docs/product-decisions.md и не обещает пользователю неподтверждённые свойства данных или сервиса.

7. Ссылки

- Видео и временные разделы: https://youtu.be/gu4pafNCXng
- Публичный код автора: https://github.com/adrianhajdin/signalist_stock-tracker-app
- Рабочий репозиторий: https://github.com/Zaywoo66/AI-Stock-market

8. Журнал изменений

- 2026-10-14, v1.0: создан единый план по видео; отдельно описаны список отслеживания и уведомления, которых нет в завершённом коде урока.